Root Finding Problem Statement

Definition:

- A *root* or *zero* of a function f(x) is a value x_r such that f(x_r) = 0 .
- Some functions, like quadratic equations, have easily identifiable roots (e.g., f(x) = x^2 - 9 has roots at x = +/- 3).

Numerical Approximation:

- For more complex functions where exact solutions are difficult to find, numerical methods like fsolve from the SciPy library can be used.
- Example:

```python
from scipy import optimize
f = lambda x: np.cos(x) - x
r = optimize.fsolve(f, -2)
```

This approach finds approximate roots, with limitations indicated by factors such as reaching a maximum number of function evaluations.

Limitations:

- Numerical methods may fail if they reach the maximum iterations without converging to a solution.
- Understanding these methods' constraints is crucial for their proper application in engineering and scientific contexts.

Success

```python
import numpy as np
from scipy import optimize

f = lambda x: np.cos(x) - x
r = optimize.fsolve(f, -2)
print("r =", r)

# Verify the solution is a root
result = f(r)
print("result=", result)
```

```
r = [0.73908513]
result= [0.]
```

Failure

```python
f = lambda x: 1/x

r, infodict, ier, mesg = optimize.fsolve(f, -2, full_output=True)
print("r =", r)

result = f(r)
print("result=", result)

print(mesg)
```

```
r = [-3.52047359e+83]
result= [-2.84052692e-84]
The number of calls to function has reached maxfev = 400.
```

Concept of Tolerance in Numerical Methods

Definition of Tolerance:

- *Tolerance* refers to the acceptable level of error in a numerical solution.
- A solution is considered to have *converged* if its error is smaller than the defined tolerance.

Error Measurement:

- Two common methods to measure error when finding roots:

1. |f(x)| : Indicates how close the function value is to zero.
2. |x_{i+1} - x_i| : Measures the difference between successive guesses, highlighting convergence speed.

Importance in Applications:

- Different applications require different levels of precision; therefore, tolerance levels must be set according to specific needs.
- Careful selection of tolerance is vital to avoid false convergence or premature stopping of the numerical method.

Practical Example of Tolerance and Root Finding

Example Analysis:

- Consider the function $f(x) = 1/x$
- Setting the error metric as $|f(x)|$ and a small tolerance value might suggest convergence even when true roots do not exist.
- It demonstrates the importance of correctly interpreting tolerance levels and error metrics.

Use of Tolerance:

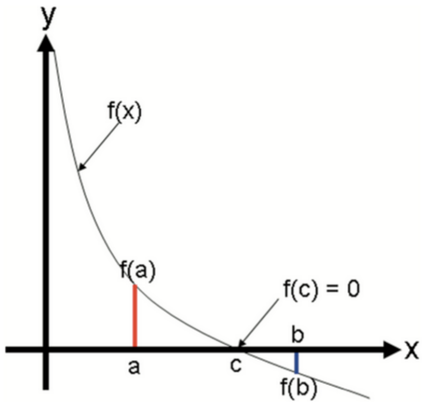
- An appropriate tolerance setting ensures that the numerical root-finding method performs optimally and provides a meaningful solution within acceptable error bounds.
- Always validate results by checking both the error magnitude and algorithm behavior.

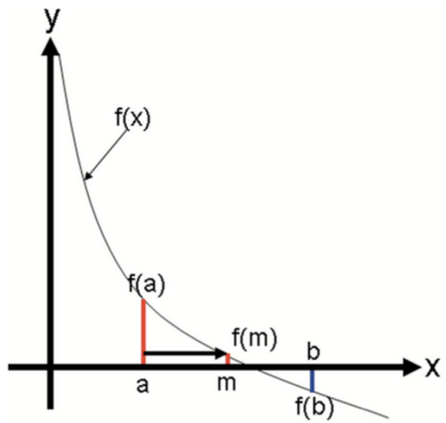
Key Takeaway:

- In numerical analysis, tolerance is not just a technical parameter but a crucial decision factor that influences the reliability and accuracy of computed results.

Bisection Method

- **Definition:**
  - The Bisection Method is a root-finding technique that applies the Intermediate Value Theorem.
  - It is used to find a root of a continuous function f(x) on an interval [a, b] where the function changes sign.
  - Key Concept:





  - If f(a) and f(b) have opposite signs ($f(a) \cdot f(b) < 0$), there exists at least one root c such that $f(c) = 0$ in the interval [a, b].
  - Process:
  - Calculate the midpoint $m = (a + b) / 2$
  - Evaluate f(m) . Depending on the sign of f(m) , update either the left bound a or the right bound b .
  - Repeat the process iteratively until the error or tolerance is sufficiently small.

Algorithm and Implementation

- **Algorithm Outline:**

1. Check Bounds: Ensure that f(a) and f(b) have opposite signs.
2. Calculate Midpoint: m = (a + b) / 2
3. Evaluate Midpoint:
   - If |f(m)| < tolerance, then m is the root.
   - If f(a) and f(m) have opposite signs, update the interval to [a, m].
   - If f(m) and f(b) have opposite signs, update the interval to [m, b].
4. Repeat: Continue until the desired accuracy is reached.

```python
import numpy as np

def my_bisection(f, a, b, tol):
    # approximates a root, R, of f bounded
    # by a and b to within tolerance
    # | f(m) | < tol with m the midpoint
    # between a and b Recursive implementation

    # check if a and b bound a root
    if np.sign(f(a)) == np.sign(f(b)):
        raise Exception(
         "The scalars a and b do not bound a root")

    # get midpoint
    m = (a + b)/2

    if np.abs(f(m)) < tol:
        # stopping condition, report m as root
        return m
    elif np.sign(f(a)) == np.sign(f(m)):
        # case where m is an improvement on a.
        # Make recursive call with a = m
        return my_bisection(f, m, b, tol)
    elif np.sign(f(b)) == np.sign(f(m)):
        # case where m is an improvement on b.
        # Make recursive call with b = m
        return my_bisection(f, a, m, tol)
```

Example and Limitations

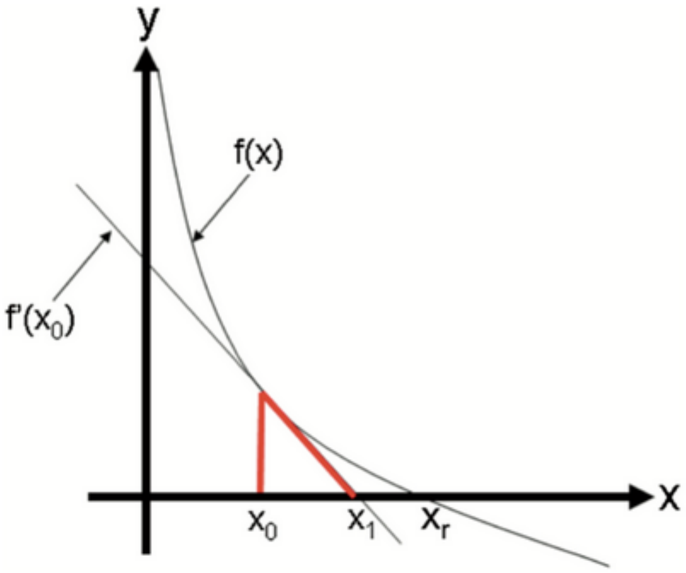
- **Example Calculation:** Function: f(x) = x^2 - 2
- **Initial Interval:** [0, 2]

- **Tolerances:**
- For tolerance 0.1, the method finds r1 = 1.4375 .
- For tolerance 0.01, it finds r01 = 1.4140625 , a closer approximation to the actual root (sqrt(2) = 1.414…….).

- **Limitations:**
- Requirement: The method requires the initial interval [a, b] to have a sign change.
- Convergence Speed: It converges linearly, which can be slower compared to other methods like Newton-Raphson.
- Exception Handling: If f(a) and f(b) do not bound a root, the method raises an error, indicating the chosen interval is incorrect.

Newton-Raphson Method

- The Newton-Raphson method is an iterative technique used to find the roots of a real-valued function.
- Assumes a smooth and continuous function f(x) with an unknown root x_r .
- Begins with an initial guess x_0 and improves upon this guess iteratively.



Linear Approximation

- The method relies on the linear approximation of the function around the initial guess x_0 .
- Uses the formula:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

- This equation represents the Newton step, used to compute an improved guess x_1 .

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

$$0 = f(x_0) + f'(x_0)(x_1 - x_0)$$

Process

1. Start with an initial guess x_0 .
2. Use the formula to find a better approximation of the root.
3. Repeat the process until the error is less than a specified tolerance level.

Implementation and Limitations of the Newton-Raphson Method

Key Limitations

- Initial Guess Sensitivity: The success of the method heavily depends on the choice of the initial guess x_0 .

- Derivative Issues: If the derivative $f'(x)$ is close to zero, the method can produce large and unstable steps.

- Root Convergence: May converge to a different root than expected or fail to converge if the initial guess is far from the actual root.

Comparison with Other Methods

- The Newton-Raphson method typically converges faster than the Bisection method when the initial guess is close to the root.

- However, due to its sensitivity, it is less reliable for functions with complex behavior.

```python
import numpy as np

f = lambda x: x**2 - 2
f_prime = lambda x: 2*x
```

```python
def my_newton(f, df, x0, tol):
    # output is an estimation of the root of f
    # using the Newton Raphson method
    # recursive implementation
    if abs(f(x0)) < tol:
        return x0
    else:
        return my_newton(f, df, x0 - f(x0)/df(x0), tol)
```

```python
estimate = my_newton(f, f_prime, 1.5, 1e-6)
print("estimate =", estimate)
print("sqrt(2) =", np.sqrt(2))
```

```
estimate = 1.4142135623746899
sqrt(2) = 1.4142135623730951
```

Using fsolve for Root Finding

- **Example Equation:**

The function used in the example:

$$f(x) = x^3 - 100x^2 - x + 100$$

Python Implementation:

```python
from scipy.optimize import fsolve

f = lambda x: x**3 - 100*x**2 - x + 100
roots = fsolve(f, [2, 80])
print(roots)
```

- **Output:** [1., 100.]

  - The initial guess of [2, 80] helps in converging to the two roots of the function: x = 1 and x = 100.

Note: fsolve admits many parameters, read about them and make sure you understand what is being done